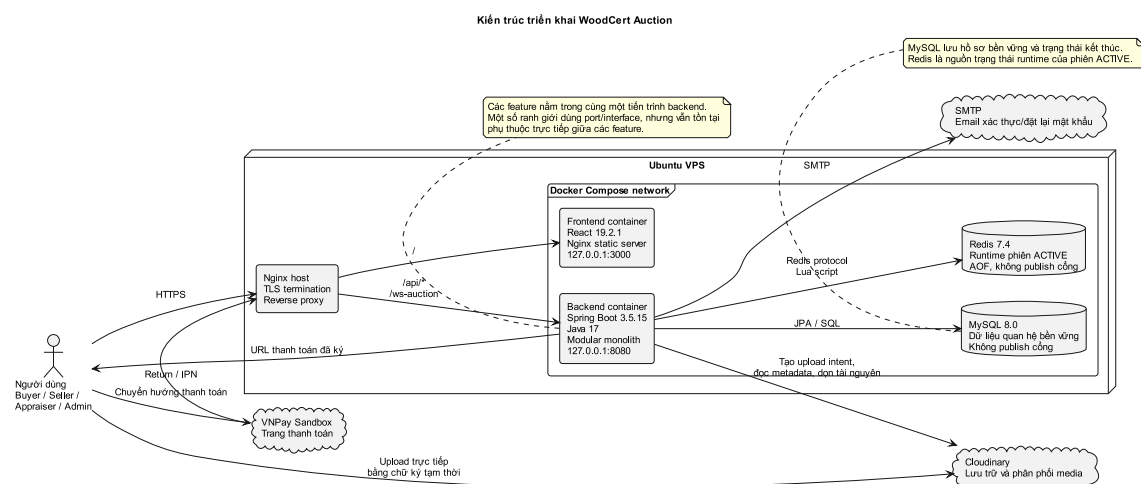


Chương này trình bày nền tảng lý thuyết và các công nghệ đang được sử dụng trong WoodCert Auction. Nội dung tập trung giải thích vai trò của từng công nghệ đối với các yêu cầu đã xác định ở Chương 2 và đối chiếu với cấu hình, mã nguồn hiện hành của dự án. Các số liệu phiên bản được lấy từ tệp khai báo phụ thuộc và cấu hình triển khai; đây không phải là kết quả đánh giá hiệu năng của hệ thống.

0.1 Tổng quan kiến trúc hệ thống

WoodCert Auction được tổ chức theo mô hình monolith mô-đun (modular monolith) ở phía máy chủ và ứng dụng trang đơn (Single Page Application – SPA) ở phía trình duyệt. Backend được triển khai trong một tiến trình Spring Boot duy nhất, nhưng mã nguồn được phân chia theo các miền nghiệp vụ như định danh, sản phẩm và kiểm định, đấu giá, tài chính, đơn hàng, giao nhận, tranh chấp và media. Một số luồng liên mô-đun sử dụng interface hoặc port để giảm phụ thuộc; tuy nhiên, hệ thống hiện vẫn có các phụ thuộc trực tiếp giữa một số feature và chưa phải kiến trúc microservices.

Mô hình này phù hợp với phạm vi đồ án vì cho phép triển khai tập trung trên một VPS, dùng chung giao dịch cơ sở dữ liệu khi cần, đồng thời vẫn duy trì ranh giới mã nguồn theo nghiệp vụ. Việc tách thành microservices chỉ được xem là định hướng mở rộng nếu sau này xuất hiện nhu cầu triển khai, mở rộng tài nguyên hoặc phát hành độc lập từng miền.



Hình 0.1: Kiến trúc triển khai của hệ thống WoodCert Auction

Nguồn: Tác giả tổng hợp từ cấu hình và mã nguồn hệ thống

Hình ?? cho thấy Nginx trên máy chủ Ubuntu tiếp nhận kết nối HTTPS và chuyển tiếp yêu cầu tới frontend hoặc backend đang bind trên loopback. MySQL và Redis chỉ nằm trong mạng Docker Compose, không công bố cổng trực tiếp ra Internet. Trình duyệt tải media trực tiếp lên Cloudinary bằng thông tin upload có

chữ ký tạm thời. Đối với VNPay Sandbox, backend tạo URL thanh toán đã ký, trình duyệt chuyển hướng sang VNPay và hệ thống tiếp nhận các yêu cầu Return/IPN để xác nhận kết quả.

0.2 Nền tảng phía Backend

0.2.1 Java 17 và Spring Boot 3.5.15

Backend sử dụng Java 17 và Spring Boot 3.5.15. Java cung cấp mô hình kiểu tĩnh, thư viện chuẩn và môi trường JVM; Spring Boot cung cấp cơ chế tự động cấu hình, dependency injection, quản lý vòng đời bean và tích hợp các thành phần trong hệ sinh thái Spring **walls2022spring, springdocs**. Trong dự án, Java record được dùng cho nhiều DTO và text block được dùng để biểu diễn Lua script, giúp giảm mã lặp và giữ cấu trúc dữ liệu rõ ràng.

Spring Data JPA đảm nhiệm ánh xạ dữ liệu quan hệ; Spring Validation kiểm tra request; Spring Mail hỗ trợ email xác thực và đặt lại mật khẩu; Spring Actuator cung cấp health endpoint; Spring Task Scheduling (@Scheduled) được cấu hình để quản lý các tác vụ nền định kỳ như mở/đóng phiên đấu giá, xử lý quá hạn thanh toán, và tự động hủy đơn hoàn tiền khi quá hạn giao hàng. Cơ chế @Transactional xác định biên giao dịch cho các thao tác như cập nhật ví, thanh toán và chuyển trạng thái. Tuy nhiên, transaction không tự loại bỏ mọi lỗi đồng thời; hệ thống còn sử dụng khóa bản ghi, optimistic locking, operation key idempotent và ràng buộc dữ liệu để kiểm soát các trường hợp tương ứng.

Các framework khác như NestJS hoặc Django cũng có thể triển khai bài toán. Spring Boot được lựa chọn vì phù hợp với mã nguồn hiện tại, hỗ trợ tốt transaction quan hệ và có hệ sinh thái kiểm thử, bảo mật, dữ liệu và vận hành được tích hợp thống nhất. Đây là lựa chọn kỹ thuật của dự án, không phải kết luận rằng các nền tảng khác không đáp ứng được tính nhất quán.

0.2.2 Spring Security và JWT HS512

JWT là định dạng biểu diễn tập claim dưới dạng JSON và được chuẩn hóa trong RFC 7519 **rfc7519**. WoodCert Auction sử dụng access token ký bằng HMAC-SHA-512 với thời gian sống 15 phút. Token này được gửi trong header cho các REST API và không được lưu bên ở frontend. Sau khi chữ ký và thời hạn token hợp lệ, hệ thống tiếp tục đối chiếu trạng thái tài khoản và lấy quyền hiệu lực từ cơ sở dữ liệu thay vì chỉ tin cậy toàn bộ quyền có trong claim.

Refresh token có thời gian sống 7 ngày, được đặt trong cookie HttpOnly, Secure, SameSite=Lax và giới hạn path /api/v1/auth. Backend chỉ lưu hash của refresh token, thực hiện rotation sau mỗi lần làm mới và có thể thu hồi phiên khi

đăng xuất hoặc đặt lại mật khẩu. Vì vậy, access-token authentication không duy trì HTTP session, nhưng vòng đời refresh token vẫn là trạng thái được quản lý trong MySQL.

Spring Security đang tắt CSRF filter mặc định cho API. Hai thao tác sử dụng refresh-token cookie là refresh và logout được bảo vệ bằng kiểm tra double-submit cookie ở lớp ứng dụng. Cơ chế chống brute-force ghi số lần đăng nhập thất bại theo email vào Redis và chặn đăng nhập tạm thời trong 15 phút sau 5 lần thất bại; cơ chế này không thay đổi lâu dài trạng thái tài khoản trong MySQL.

Giải pháp tự quản lý xác thực giúp dự án kiểm soát trực tiếp token rotation, capability và RBAC. Đối lại, backend phải tự chịu trách nhiệm bảo vệ khóa HMAC, xử lý thu hồi token và kiểm tra quyền ở từng endpoint. Các nền tảng IAM như Keycloak hoặc Auth0 là lựa chọn phù hợp khi cần liên kết danh tính, quản trị tập trung hoặc nhiều ứng dụng dùng chung.

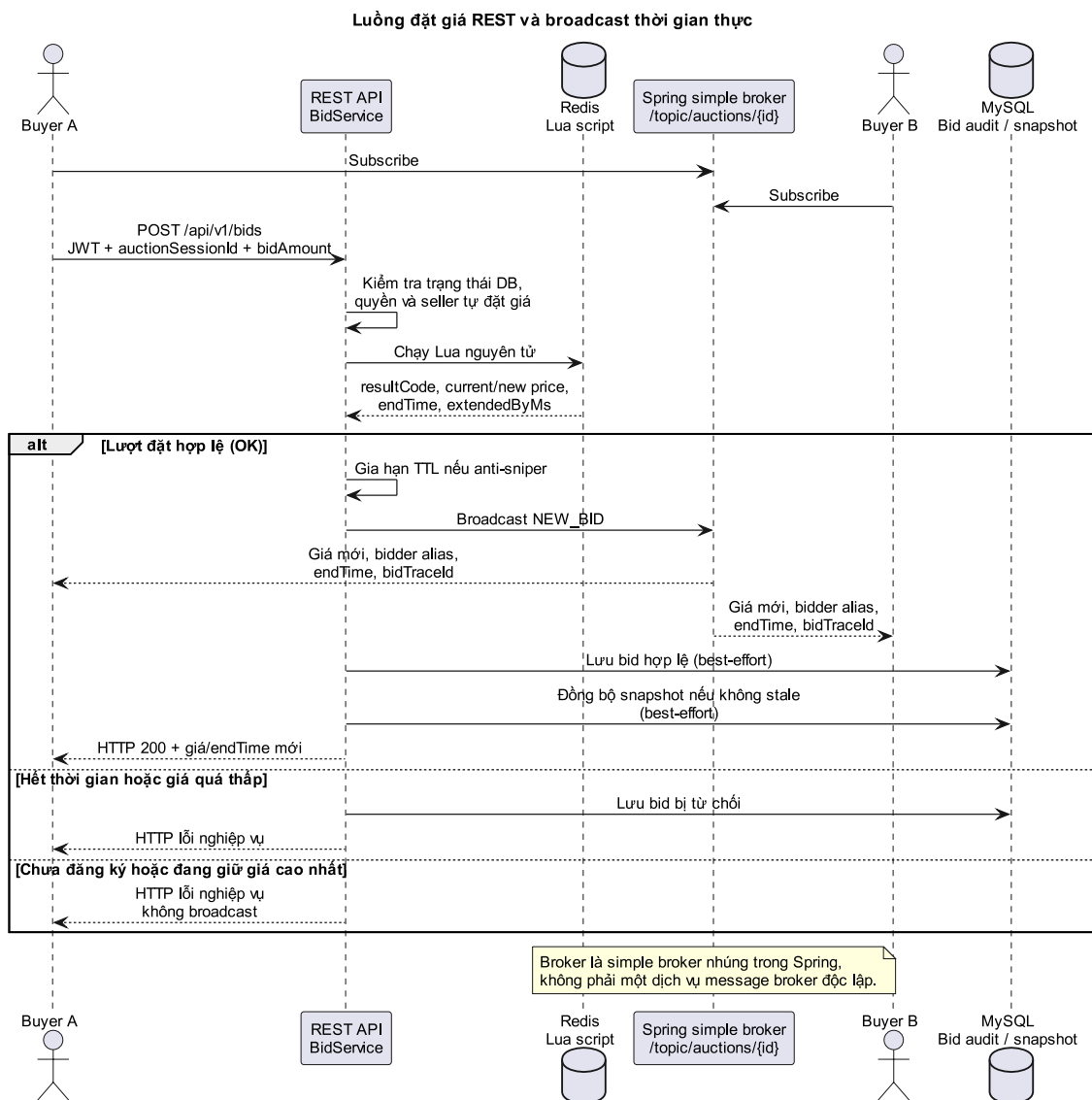
0.2.3 WebSocket, STOMP và SockJS

WebSocket cung cấp kênh truyền thông hai chiều trên một kết nối lâu dài và được chuẩn hóa trong RFC 6455 **rfc6455**. Spring WebSocket kết hợp STOMP để tổ chức các subscription theo topic; SockJS cung cấp cơ chế fallback khi môi trường mạng không thiết lập được WebSocket trực tiếp **springwebsocketsdocs**.

Trong WoodCert Auction, lệnh đặt giá không được gửi qua STOMP. Buyer gửi POST /api/v1/bids để đi qua xác thực và kiểm tra nghiệp vụ của REST API. Sau khi Redis chấp nhận lượt đặt, backend dùng simple broker nhúng trong Spring để phát sự kiện NEW_BID tới topic của phiên. Kênh subscribe hiện là read-only broadcast và payload công khai sử dụng bí danh đã che của người đặt giá.

Như thể hiện tại Hình ??, Redis/Lua là biên quyết định chấp nhận bid. Với bid hợp lệ, backend phát sự kiện trước rồi mới lưu bid và snapshot MySQL theo cơ chế best-effort trong cùng luồng request. Bid hết thời gian hoặc thấp hơn mức tối thiểu được ghi nhận là bid bị từ chối; trường hợp chưa đăng ký hoặc đang giữ giá cao nhất trả lỗi mà không broadcast.

HTTP polling và Server-Sent Events đều là phương án có thể dùng cho luồng server-to-client. STOMP/WebSocket được giữ lại vì frontend đã có cơ chế subscribe dùng chung cho phòng đấu giá của Buyer và màn hình theo dõi của Seller, đồng thời Nginx đã được cấu hình chuyển tiếp WebSocket. Do hệ thống hiện chỉ broadcast một chiều, lợi ích chính nằm ở kết nối lâu dài và mô hình topic, không phải ở việc gửi bid hai chiều qua WebSocket.



Hình 0.2: Luồng xử lý đặt giá và phát sự kiện thời gian thực

Nguồn: Tác giả tổng hợp từ BidService và cấu hình WebSocket

0.3 Quản lý và lưu trữ dữ liệu

0.3.1 Flyway

Flyway quản lý thay đổi schema bằng các migration có phiên bản và lưu lịch sử thực thi trong database **flywaydocs**. Khi ứng dụng khởi động, Flyway áp dụng các migration chưa chạy và kiểm tra checksum của migration đã ghi nhận. Checksum giúp phát hiện nội dung migration bị thay đổi sau khi áp dụng, nhưng không tự xác định thay đổi đó có được phê duyệt hay không.

Schema hiện hành của dự án được hình thành từ bốn migration: `V1__baseline_schema.sql`, `V2__seed_reference_data.sql`, `V3__seed_demo_users.sql` và `V4__add_dispute_conversation.sql`. Cách quản lý này bảo đảm cấu trúc bảng, khóa ngoại, chỉ mục và dữ liệu tham chiếu có thể được tái tạo theo cùng thứ tự giữa các môi trường. So với thay đổi schema thủ công, migration có phiên bản dễ truy vết và có thể kiểm tra trong CI; Liquibase là lựa chọn tương đương nếu dự án cần changelog đa định dạng hoặc cơ chế rollback khai báo.

0.3.2 MySQL 8.0

MySQL 8.0 là nơi lưu trữ dữ liệu quan hệ bền vững: người dùng và quyền, sản phẩm, báo cáo kiểm định, phiên đấu giá, bid audit, ví và giao dịch tài chính, đơn hàng, giao nhận, tranh chấp và nhật ký quản trị **mysqldocs**. Media binary không nằm trong MySQL mà được lưu ở Cloudinary; MySQL chỉ lưu metadata và quan hệ nghiệp vụ. Trạng thái biến đổi nhanh của phiên `ACTIVE` nằm ở Redis, sau đó trạng thái kết thúc chính thức được ghi về MySQL.

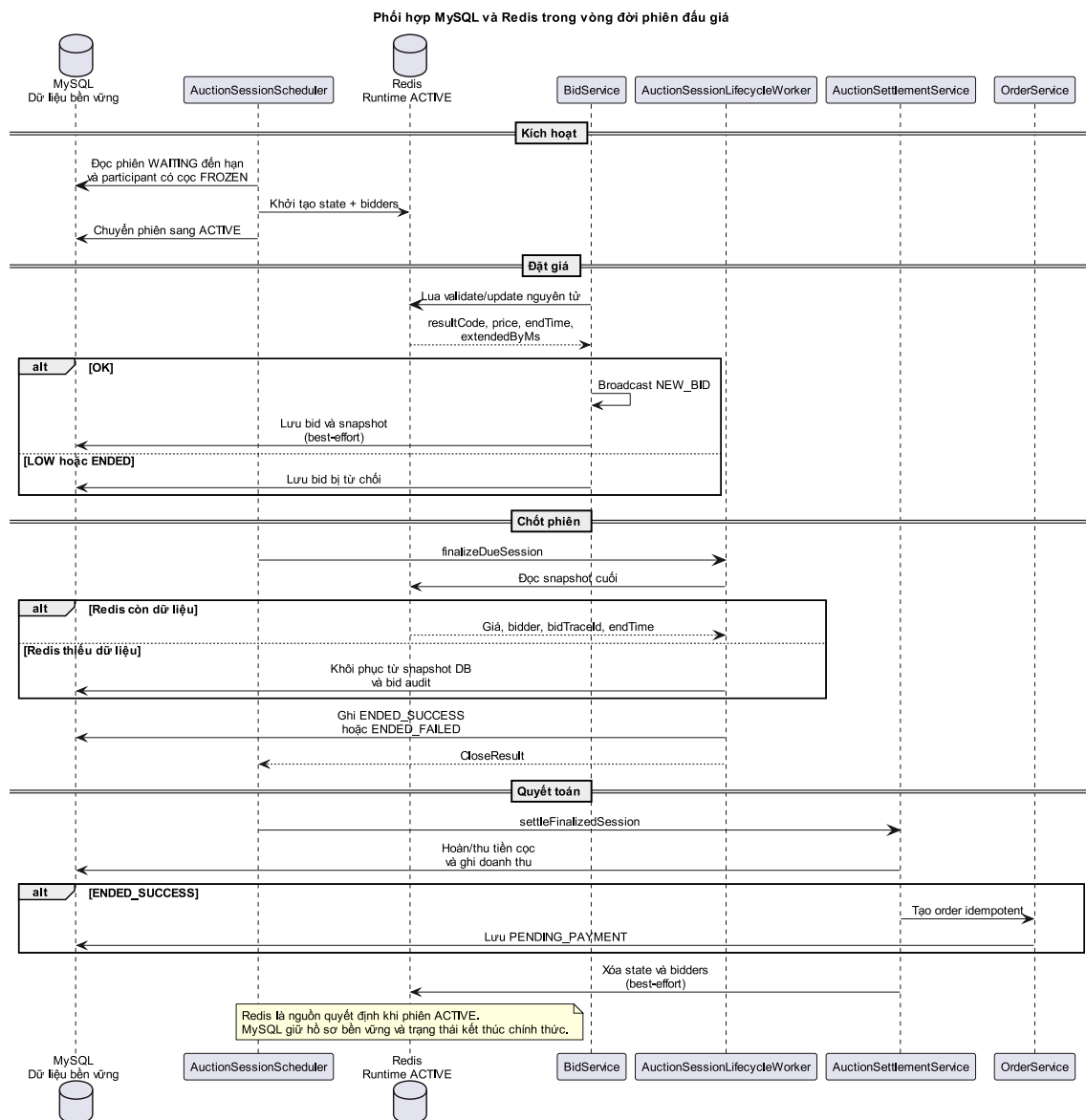
InnoDB hỗ trợ transaction, khóa và ràng buộc khóa ngoại. Tính đúng của số dư không chỉ dựa vào ACID mà còn dựa vào operation key idempotent, khóa cập nhật ví, optimistic locking và transaction ở service. PostgreSQL cũng có thể đáp ứng yêu cầu quan hệ; MySQL được giữ vì schema, migration, Docker Compose và repository hiện tại đã được xây dựng, kiểm thử trên nền tảng này.

0.3.3 Redis 7.4 và Lua script

Redis 7.4 lưu trạng thái runtime của phiên đang hoạt động dưới dạng Hash và Set **redisdocs**. Hash chứa giá hiện tại, bước giá, người giữ giá cao nhất và mốc kết thúc; Set chứa danh sách user đã đăng ký và có tiền cọc ở trạng thái phù hợp. Countdown trên giao diện được tính từ `endTimeEpochMs`; Redis không lưu một bộ đếm giảm dần riêng.

Lua script thực hiện liên tiếp các kiểm tra thời gian, tư cách tham gia, người đang giữ giá cao nhất, bước giá và cập nhật anti-sniper trong một lần thực thi nguyên tử.

Tính nguyên tử ngăn các lệnh Redis trong script bị xen kẽ bởi request khác; các lỗi ở hệ thống ngoài Redis vẫn phải được xử lý riêng. Production bật AOF để tăng khả năng phục hồi dữ liệu Redis sau khi tiến trình khởi động lại.



Hình 0.3: Phối hợp MySQL và Redis trong vòng đời phiên đấu giá

Nguồn: Tác giả tổng hợp từ các dịch vụ auction runtime và settlement

Hình ?? phân biệt ba giai đoạn. Lớp lập lịch AuctionSessionScheduler (sử dụng cơ chế @Scheduled của Spring) tải phiên đến hạn và participant từ MySQL để khởi tạo runtime Redis. Trong giai đoạn ACTIVE, Lua script là nguồn quyết định bid, còn MySQL nhận bản lưu audit và snapshot best-effort. Khi đến hạn, lifecycle worker ưu tiên snapshot Redis nhưng có đường phục hồi từ snapshot và bid audit trong MySQL; worker ghi ENDED_SUCCESS hoặc ENDED_FAILED trước khi settlement xử lý tiền cọc, tạo đơn hàng và dọn Redis.

0.3.4 Cloudinary

Cloudinary cung cấp lưu trữ, biến đổi và phân phối media qua CDN **cloudinarydocs**. Backend tạo upload intent có chữ ký, còn trình duyệt upload trực tiếp để payload ảnh không đi qua VPS. Sau upload, backend đọc metadata thực tế từ Cloudinary, kiểm tra loại và kích thước, sau đó chuyển `media_assets` sang `ACTIVE`.

Các usage đã triển khai gồm ảnh đại diện, ảnh sản phẩm, ảnh chứng minh kiểm định và bằng chứng tranh chấp. Ảnh sản phẩm là bắt buộc khi tạo sản phẩm; ảnh chứng minh do Appraiser tải lên trong báo cáo kiểm định là tùy chọn. Hệ thống có tác vụ dọn tài nguyên `PENDING` quá 24 giờ và tài nguyên mồ côi. Video đóng gói giao hàng mới là phạm vi dự kiến, không được trình bày như chức năng hiện hành.

Lưu file trên VPS hoặc dùng object storage như Amazon S3 đều là phương án thay thế. Cloudinary được lựa chọn vì quy trình hiện tại cần upload có chữ ký, metadata verification, URL phân phối và biến đổi ảnh trong cùng một dịch vụ.

0.4 Công nghệ phía Frontend

0.4.1 React 19.2.1, TypeScript 5.9.3 và Vite 7.2.6

React xây dựng giao diện theo component và cơ chế render khai báo **reactdocs**. TypeScript bổ sung kiểm tra kiểu tĩnh cho JavaScript **bierman2014typescript**. Vite sử dụng native ES modules trong môi trường phát triển, hỗ trợ Hot Module Replacement và thực hiện build production theo pipeline bundling của Vite **vitedocs**.

Ba công nghệ này được dùng để xây dựng SPA theo các khu vực public, buyer, seller, appraiser và admin. TypeScript giúp phát hiện sai lệch kiểu tại thời điểm build, nhưng dữ liệu nhận từ API vẫn cần được kiểm soát bằng contract và xử lý lỗi runtime. Cấu hình Vite chia bundle theo nhóm phụ thuộc, còn các route được lazy-load để tránh tải toàn bộ màn hình ngay ở lần truy cập đầu.

Angular và Vue là các lựa chọn khả thi. React được giữ vì codebase, component và hệ sinh thái kiểm thử của dự án đã được tổ chức quanh React; quyết định này không dựa trên khẳng định chung rằng React luôn nhẹ hoặc nhanh hơn các framework khác.

0.4.2 TanStack Query 5.90.12 và Zustand 5.0.9

TanStack Query quản lý server state, cache và vòng đời request trong ứng dụng React **reactquerydocs**. Dự án cấu hình query đọc dữ liệu thử lại một lần khi lỗi, trong khi mutation không tự retry để tránh lặp thao tác tài chính hoặc thay đổi trạng thái. Cache được làm sạch khi đăng xuất hoặc khi danh tính người dùng thay đổi, hạn chế dữ liệu của phiên trước xuất hiện trong phiên sau.

Zustand quản lý access token trong bộ nhớ và trạng thái xác thực cục bộ **zustanddocs**. Selector cho phép component chỉ subscribe phần state cần thiết. React Context hoặc Redux Toolkit đều có thể đáp ứng; Zustand được lựa chọn vì phạm vi global client state hiện tương đối nhỏ, còn dữ liệu nghiệp vụ từ server đã được TanStack Query quản lý.

0.4.3 Tailwind CSS 4.1.17 và Radix UI

Tailwind CSS cung cấp utility class để xây dựng layout và style theo design token **tailwinddocs**. Radix UI cung cấp primitive không áp đặt style và tập trung vào hành vi tương tác, bàn phím và thuộc tính hỗ trợ tiếp cận **radixdocs**. Dự án hiện sử dụng một số primitive cụ thể như Dialog, Dropdown Menu, Label và Slot; không xem Radix là bộ component hoàn chỉnh cho toàn bộ giao diện.

Bootstrap hoặc các component library có giao diện dựng sẵn có thể giúp phát triển nhanh hơn. Tailwind kết hợp Radix phù hợp với codebase hiện tại vì cho phép kiểm soát style riêng trong khi tái sử dụng hành vi tương tác của các primitive cần thiết.

0.5 Hạ tầng triển khai, CI/CD và kiểm thử

0.5.1 Nginx và Let's Encrypt/Certbot

Nginx là reverse proxy trên host VPS và cũng được dùng trong frontend container để phục vụ static build **nginxdocs**. Cấu hình host chuyển `/api/` và `/ws-auction` tới backend loopback, các đường dẫn còn lại tới frontend loopback; WebSocket được chuyển tiếp bằng Upgrade header với `proxy_read_timeout 3600s`. Tập Nginx trong repository khai báo điểm bắt đầu HTTP, còn quy trình triển khai dùng Certbot để cấu hình chứng chỉ Let's Encrypt và HTTPS trên máy chủ **certbotdocs**.

0.5.2 Docker và Docker Compose

Backend, frontend, MySQL và Redis được đóng gói thành các image/container riêng; Docker Compose mô tả mạng, volume, health check và phụ thuộc khởi động **dockerdocs**, **composedocs**. Cổng MySQL và Redis không được publish ra host. Backend và frontend chỉ bind vào địa chỉ loopback để mọi request từ Internet phải đi qua Nginx.

Container giúp cố định môi trường chạy và giảm khác biệt giữa các lần triển khai, nhưng không tự bảo đảm mọi máy phát triển hoàn toàn giống nhau. Kubernetes phù hợp khi cần điều phối nhiều node hoặc nhiều workload độc lập; với một VPS và số container hiện tại, Docker Compose có chi phí vận hành thấp hơn.

0.5.3 GitHub Actions và triển khai production

Workflow CI chạy Maven test cho backend; lint, typecheck, Vitest và build cho frontend; Playwright E2E; kiểm tra script triển khai, cấu hình Docker Compose và build hai Docker image **actionsdocs**. CI không chạy Prettier. Workflow release tạo và đẩy image lên GitHub Container Registry sau khi CI thành công trên nhánh chính.

Job deploy chỉ chạy khi biến repository `CD_ENABLED` bằng `true`. Job sử dụng GitHub Environment `production` và môi trường này được cấu hình bắt buộc phê duyệt thủ công. Sau khi được duyệt, workflow sao chép deployment script lên VPS qua SCP, thực thi script qua SSH và kiểm tra public readiness endpoint. Secret ứng dụng không được đưa vào repository hoặc GitHub Actions mà tiếp tục nằm trong tệp cấu hình production trên VPS.

0.5.4 Công nghệ kiểm thử

Backend sử dụng JUnit 5 cho unit test và integration test; Testcontainers khởi tạo MySQL/Redis hoặc hạ tầng Docker cần thiết cho các kiểm thử wiring và migration **junitdocs**, **testcontainersdocs**. Frontend sử dụng Vitest và Testing Library cho logic/component, còn Playwright kiểm tra luồng trình duyệt đầu cuối **vitestdocs**, **playwrightdocs**. Việc test tồn tại trong codebase không đồng nghĩa mọi hành vi production đã được kiểm chứng; báo cáo kết quả kiểm thử phải dựa trên command hoặc report được chạy ở phiên bản tương ứng.

Các chỉ tiêu p95 và số người dùng đồng thời nêu ở Chương 2 là mục tiêu thiết kế chưa được benchmark trong giai đoạn này. Để xác nhận, Chương 4 cần mô tả môi trường, commit, dữ liệu khởi tạo và kịch bản đo riêng cho REST bid và thời gian từ lúc bid được chấp nhận tới lúc subscriber nhận `NEW_BID`. Mục tiêu availability theo tháng chỉ có thể đánh giá bằng dữ liệu giám sát đủ thời gian, không thể suy ra từ một lần chạy test.

0.6 Tổng kết chương

Chương 3 đã trình bày các nền tảng đang tạo nên WoodCert Auction và vai trò của chúng đối với yêu cầu nghiệp vụ. Spring Boot và MySQL quản lý logic và dữ liệu bền vững; Redis/Lua xử lý trạng thái phiên `ACTIVE`; WebSocket/STOMP phân phối cập nhật; React và các thư viện frontend tổ chức SPA; Docker, Nginx và GitHub Actions hỗ trợ triển khai có kiểm soát. Các sơ đồ trong chương phản ánh trạng thái source hiện hành và làm cơ sở cho phần phân tích thiết kế, triển khai và đánh giá ở Chương 4.